

# JWT Implementation

## Generate JWT Token

### Step 1:

Install the required packages:

**Microsoft.AspNetCore.Authentication.JwtBearer;**

**Microsoft.IdentityModel.Tokens;**

**System.IdentityModel.Tokens.Jwt;**

**System.Security.Claims**

### Step 2:

In appSetting.json file add the following:

```
"Jwt": {  
  "Key": "ThisisyourkeyThisisyourkeyThisisyourkeyThisisyourkeyThisisyourkey",  
  "Issuer": "Iamyourissuer",  
  "Audience": "Iamyouraudience",  
  "Subject": "thisisyoursubject"  
}
```

Step 3: Create Claims with Subject and Payload.

```
var claims = new[]  
{  
  new Claim(System.IdentityModel.Tokens.Jwt.JwtRegisteredClaimNames.Sub,_configuration["Jwt:Subject"]),  
  new Claim("Id",username),  
  new Claim("Password",password),  
  new Claim(ClaimTypes.Role,"Admin")  
};  
  
var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]));  
var signIn = new SigningCredentials(key,SecurityAlgorithms.HmacSha256);
```

```

var token = new JwtSecurityToken(
    _configuration["Jwt:Issuer"],
    _configuration["Jwt:Audience"],
    claims,
    expires: DateTime.UtcNow.AddMinutes(1),
    signingCredentials: signIn
);
return new JwtSecurityTokenHandler().WriteToken(token)

```

### Validate JWT Token in program.cs file

Namespace required:

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
```

Add following code before builder.build()

```

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(
    options =>
    {
        options.RequireHttpsMetadata = false;
        options.SaveToken = true;
        options.TokenValidationParameters = new TokenValidationParameters()
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidAudience = builder.Configuration["Jwt:Audience"],
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            IssuerSigningKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]))
        };
    });

```

# On ActionResult Method

## [Authorize(Roles = "Admin")]

### appsetting.json

```
{  "Jwt": {    "Key": "ThisisyourkeyThisisyourkeyThisisyourkeyThisisyourkeyThisisyourkeyThisisyourkeyThisisyourkeyThisisyourkey",    "Issuer": "Iamyourissuer",    "Audience": "Iamyouraudience",    "Subject": "thisisyoursubject"  }},
```

### Create one service to generate Token

```
using Microsoft.IdentityModel.Tokens;
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Reflection.Metadata.Ecma335;
using System.Security.Claims;
using System.Text;
```

```
2 references
public class AuthService : IAuthService
{
    private const string username = "khalid";
    private const string password = "khalid";
    private IConfiguration _configuration;
    0 references
    public AuthService(IConfiguration configuration)
    {
        _configuration = configuration;
    }
    2 references
    public string Login(LoginRequests loginRequests)
    {
        var username = loginRequests.Username;
        var claims = new[]
        {
            new Claim(System.IdentityModel.Tokens.Jwt.JwtRegisteredClaimNames.Sub, _configuration["Jwt:Subject"]),
            new Claim("Id", username),
            new Claim("Password", password),
            new Claim(ClaimTypes.Role, "Admin")
        };
        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]));
        var signIn = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
        var token = new JwtSecurityToken(
            _configuration["Jwt:Issuer"],
            _configuration["Jwt:Audience"],
            claims,
            expires: DateTime.UtcNow.AddMinutes(60),
            signingCredentials: signIn
        );
        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}
```

## Validate Token in program.cs

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi.Models;
using System.Text;

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(
    options =>
    {
        options.RequireHttpsMetadata = false;
        options.SaveToken = true;
        options.TokenValidationParameters = new TokenValidationParameters()
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidAudience = builder.Configuration["Jwt:Audience"],
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]))
        };
    });
```

## In Controller

```
[HttpGet]
[ServiceFilter(typeof(ResultFilter))]
[Authorize(Roles = "Admin")]
0 references
public async Task<ActionResult<List<Student>>> GetStudents()
{
    var data = await context.Students.ToListAsync();
    return Ok(data);
}

[HttpGet("{id}")]
[Authorize(Roles = "User")]
0 references
public async Task<ActionResult<List<Student>>> GetStudentById(int id)
{
    var student = await context.Students.FindAsync(id);
    if(student==null)
    {
        return NotFound();
    }
    return Ok(student);
}
```